



# **Metric instrumentation: from Dropwizard to OpenTelemetry**

---

**Paul Hsieh**

**Principal Engineer,  
Infrastructure Group,  
Taboola**

**Taboola delivers  
performance at  
scale beyond  
search and social.**

**11 K+**

Global  
Publishers

**2 K+**

Employees  
- 500 in R&D  
- 35 Global Offices

**600M+**

Daily Active  
Users

**130 M+**

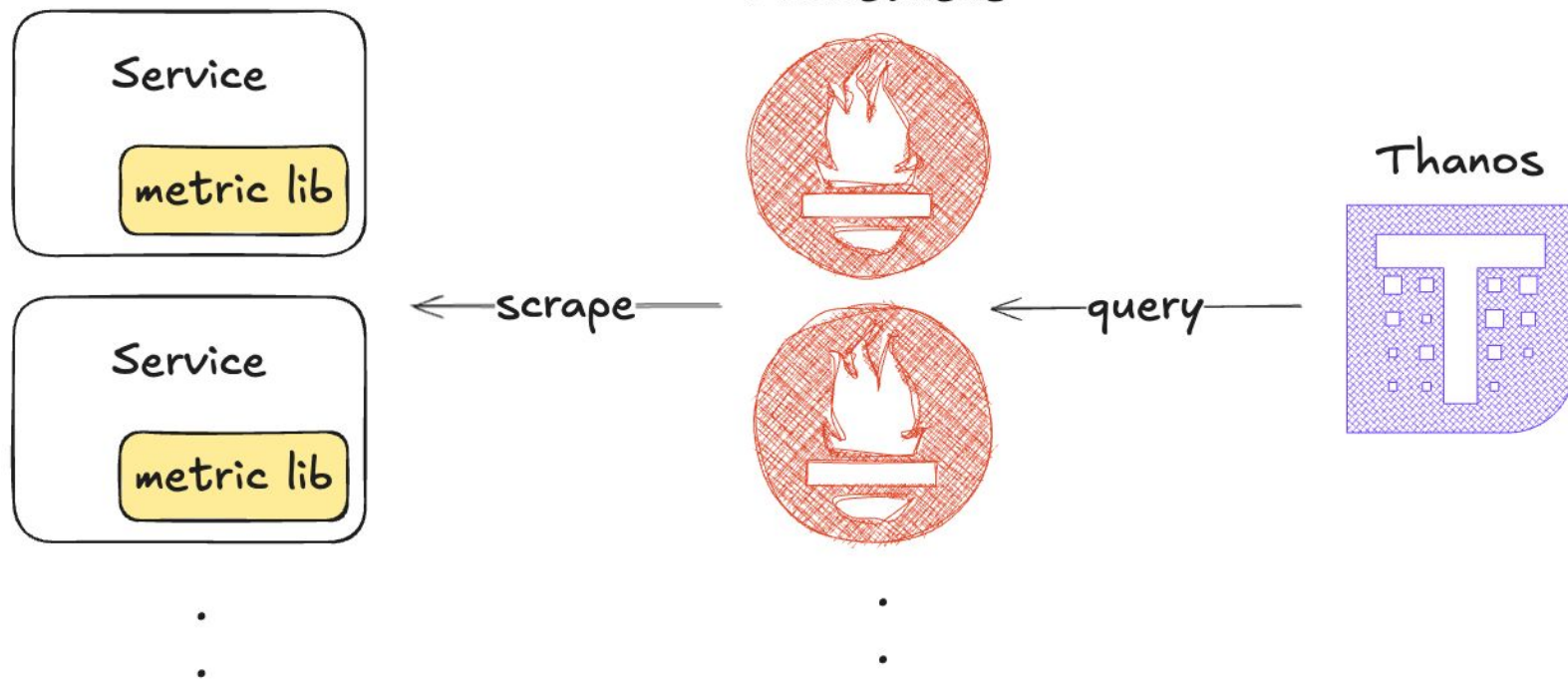
Annual R&D  
Investment

**18 K+**

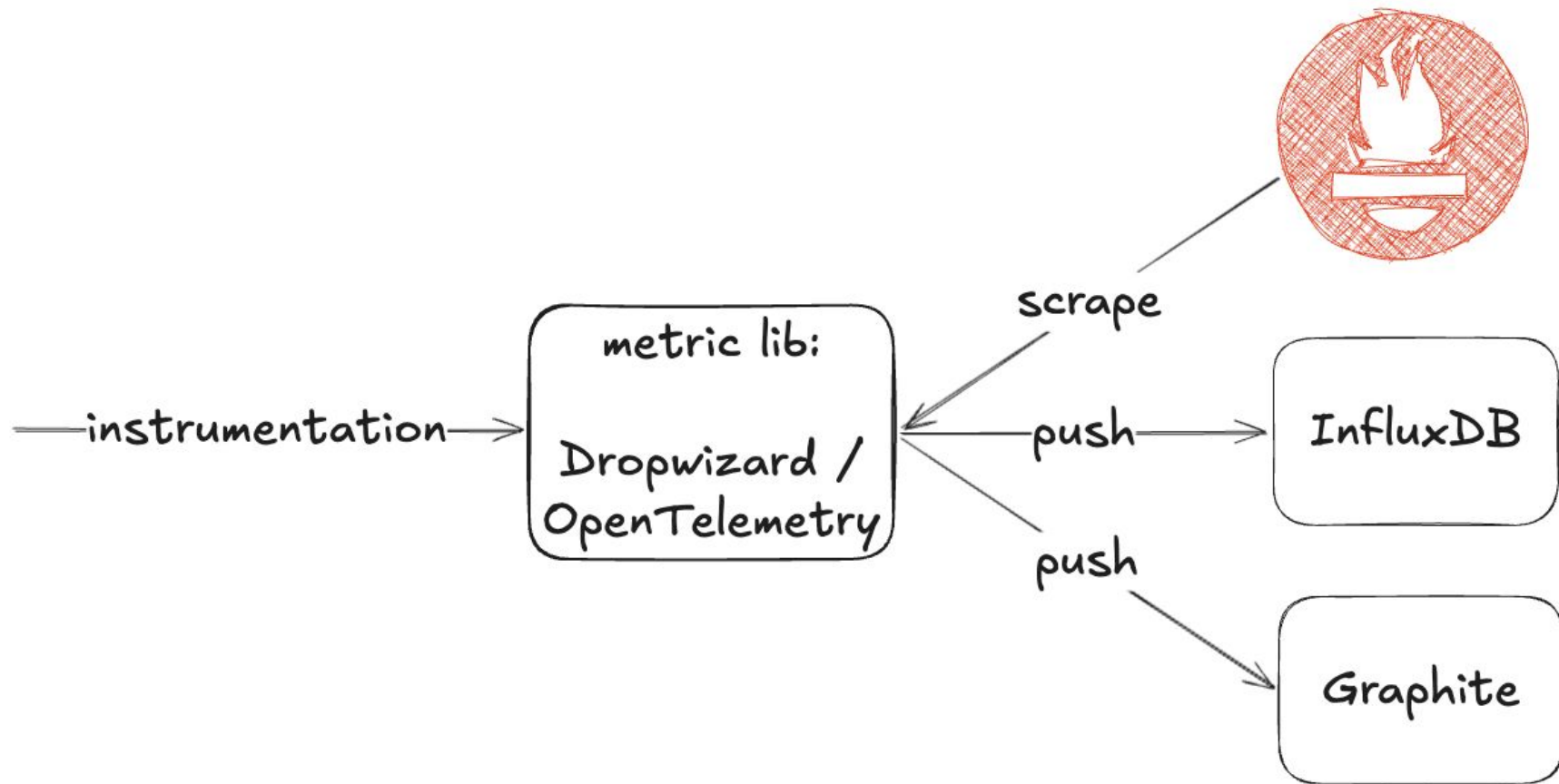
Global  
Advertisers

# Metric topology

- 1B+ metrics/m

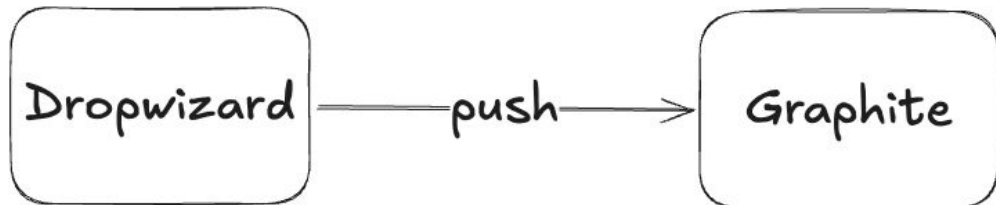


# Metric library

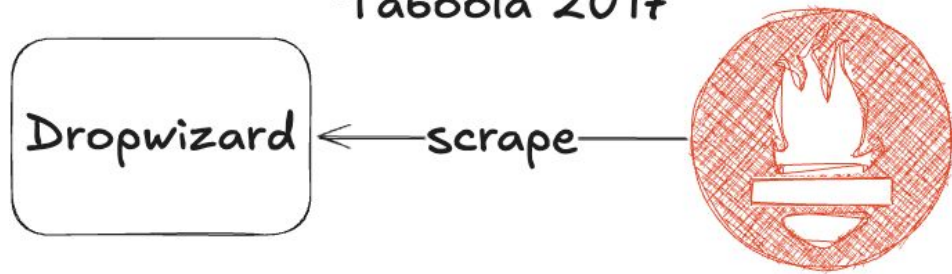


# Dropwizard history

Taboola 2014



Taboola 2017



# Motivation

- Aggregatable histogram
- Semantic names and labels
- Recording rule scalability
- Exemplar integration

# Motivation – Aggregatable histogram

- Dropwizard exports summary metric
  - Client calculated quantiles
  - EX: `latency_p99{instance="web-0"}`
- Good precision 👍



# Motivation – Aggregatable histogram

- What is overall p99 latency?
  - `latency_p99{instance="web-0"} 1`
  - `latency_p99{instance="web-1"} 10`

# Motivation – Aggregatable histogram

- What is overall p99 latency?
  - `latency_p99{instance="web-0"} 1`
  - `latency_p99{instance="web-1"} 10`
- Average? What if –
  - `web-0` handled 100 requests
  - `web-1` handled 1 request

# Motivation – Aggregatable histogram

- What is overall p99 latency?

- `latency_p99{instance="web-0"} 1`

- `latency_p99{instance="web-1"} 10`

- Weighted average? 🤔

`sum(count * p99) / total_count`

# Motivation – Semantic name and labels

- Dropwizard metrics are dot-separated string

- EX: `requests.count`

- Taboola's Prometheus mapping –

- ```
SERVICE_counter{_label_0="requests", _label_1="count"}
```

# Motivation – Recording rule scalability

- Scalability bottleneck on generic names
  - `SERVICE_counter`
  - `SERVICE_histogram_p99`
  - ...
- `record: SERVICE_counter_total:rate`  
`expr: sum without (instance) (rate(SERVICE_counter[5m]))`

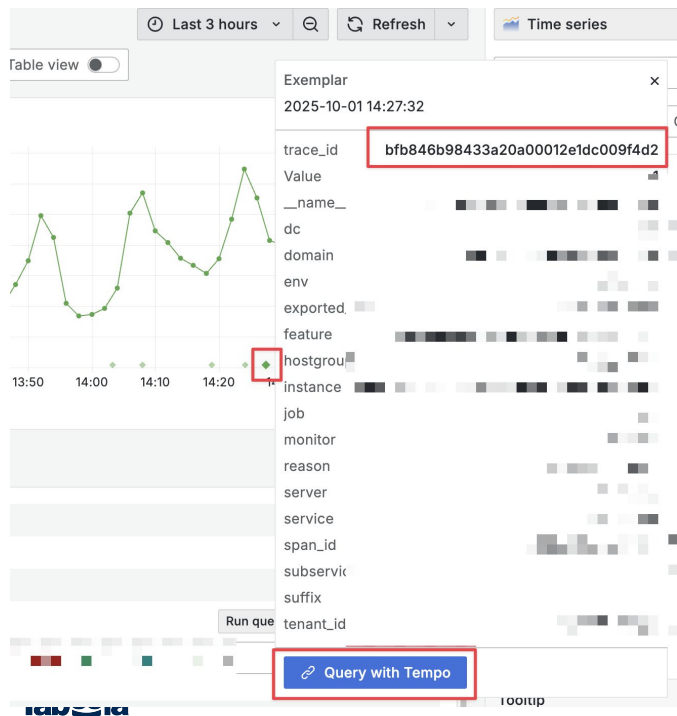
# Motivation – Exemplar integration

- Dive from metrics to traces



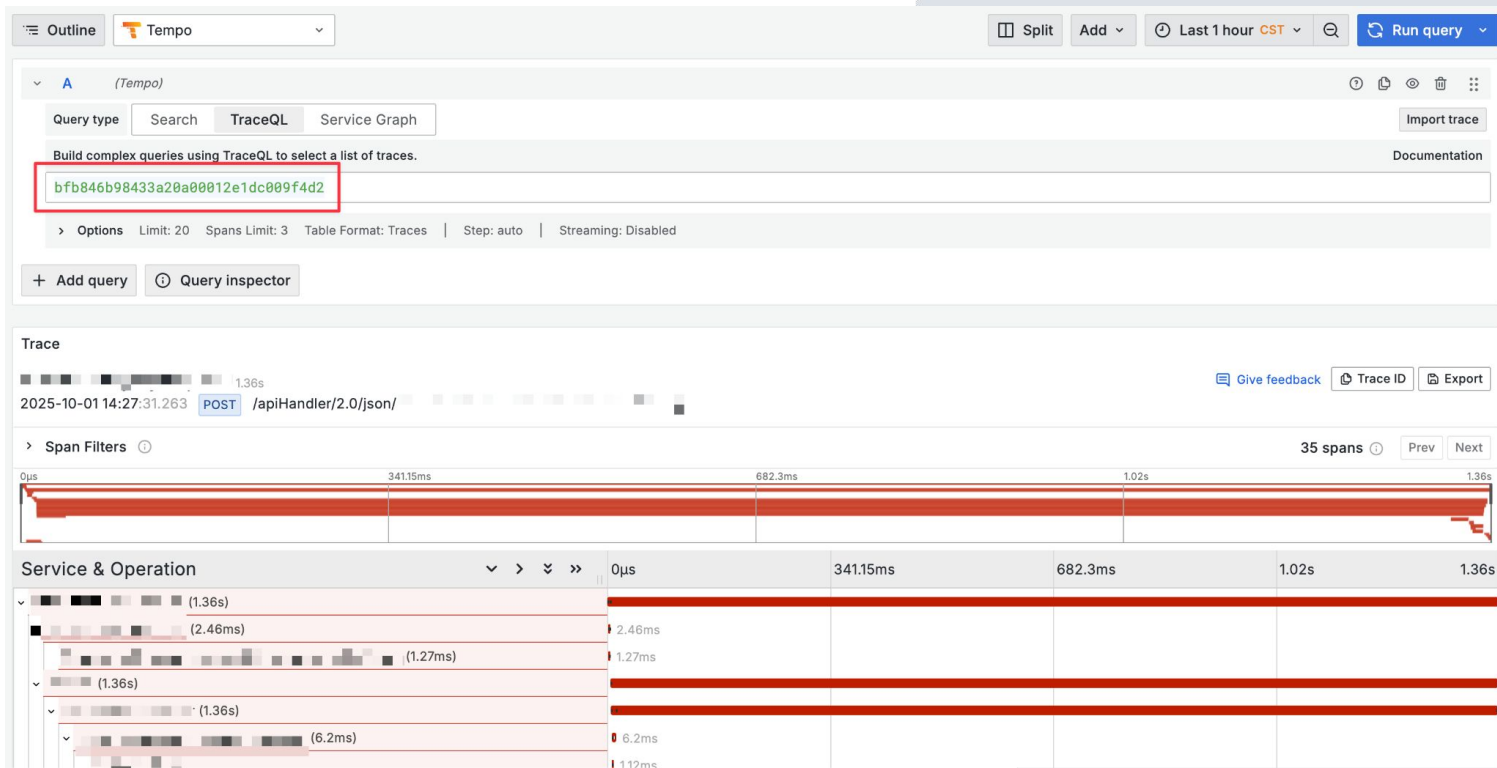
## Motivation – Exemplar integration

- Dive from metrics to traces



# Motivation – Exemplar integration

- Dive from metrics to traces





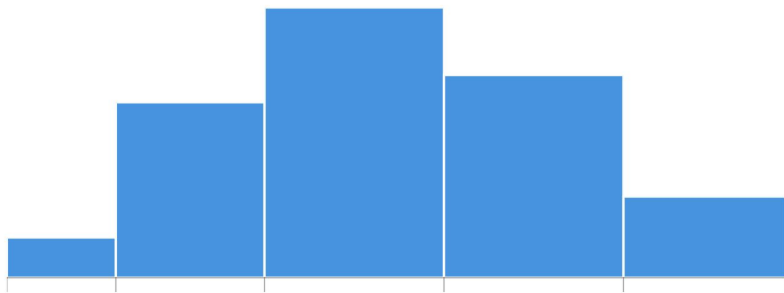
# Motivation to solution

- Aggregatable histogram =>
  - Lib supporting classic or native histogram
  - Query time calculated quantiles
  - Aggregatable across instances

# Prometheus histogram

## Classic histogram

You pick the buckets



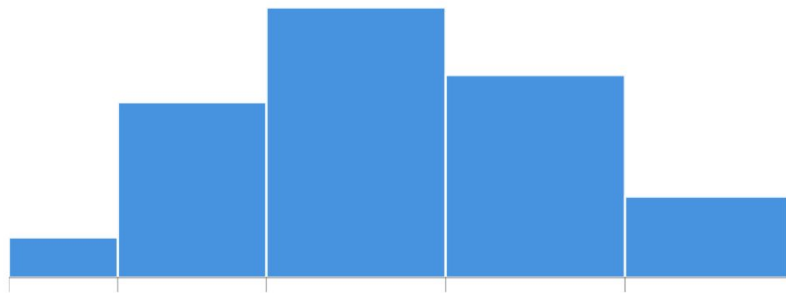
~10 fixed buckets

**10+ time series**

# Prometheus histogram

## Classic histogram

You pick the buckets

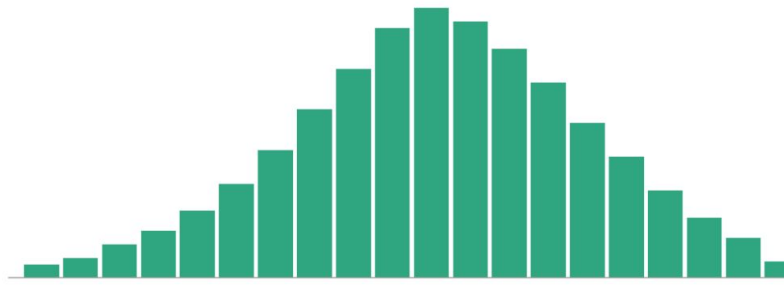


~10 fixed buckets

**10+ time series**

## Native histogram

Buckets auto-scale



~160 auto buckets

**1 time series**

# Motivation to solution

- Semantic names and labels =>
  - Lib supporting name and labels
- Recording rule scalability =>
  - Lib supporting name and labels

# Recording rule scalability

- `SERVICE_counter{ _label_0="getInfo" }`
- `SERVICE_counter{ _label_0="getMetadata" }`
- `record: SERVICE_counter_total:rate`  
`expr: sum without (instance) (rate(SERVICE_counter[5m]))`

=>

- `get_info_total{ }`
- `get_metadata_total{ }`

# Motivation to solution

- Exemplar integration =>
  - Lib supporting exemplars

# OpenTelemetry

- Support all use cases
  - Classic and **native** histogram
  - Name and labels
  - Exemplar integration
- Active development and community
- Already used for tracing in Taboola

# Requirement

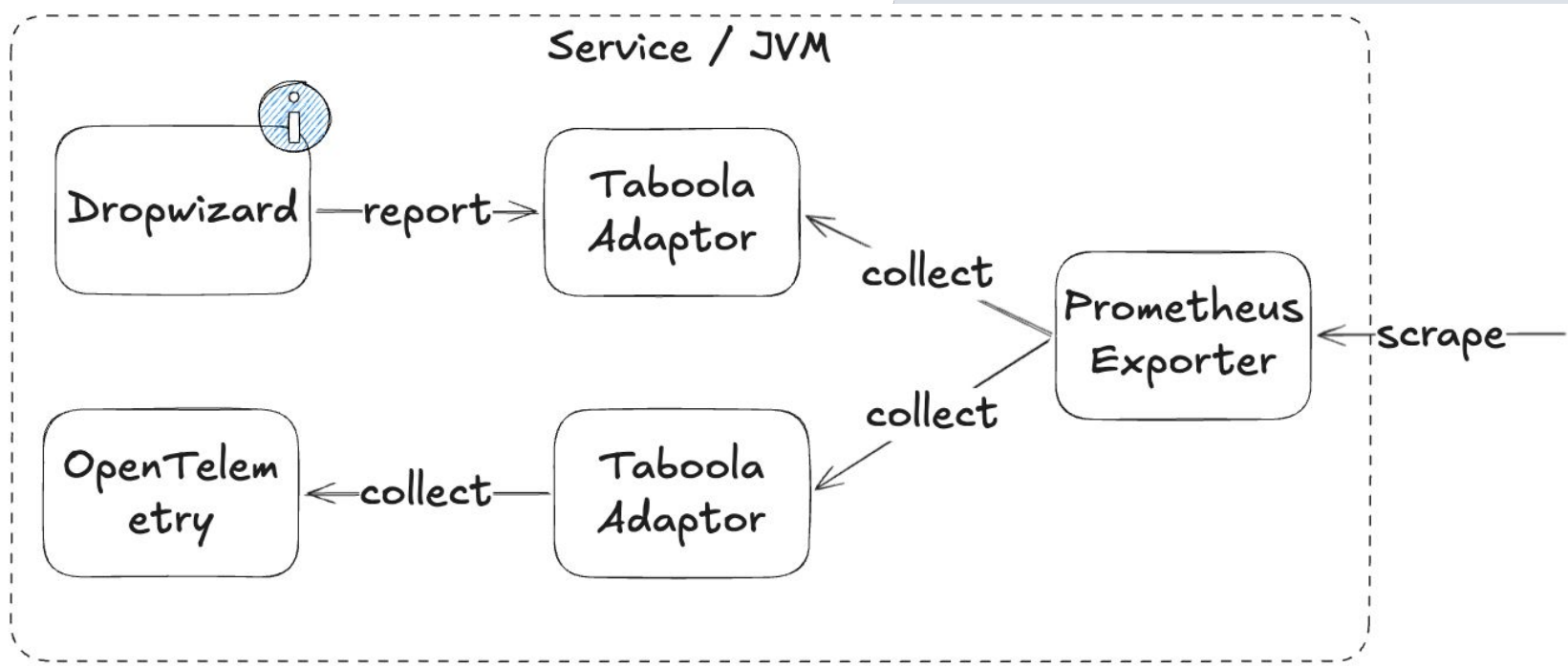
- Transparent migration
- Backward compatibility
- Parallel verification
- Ease of rollback



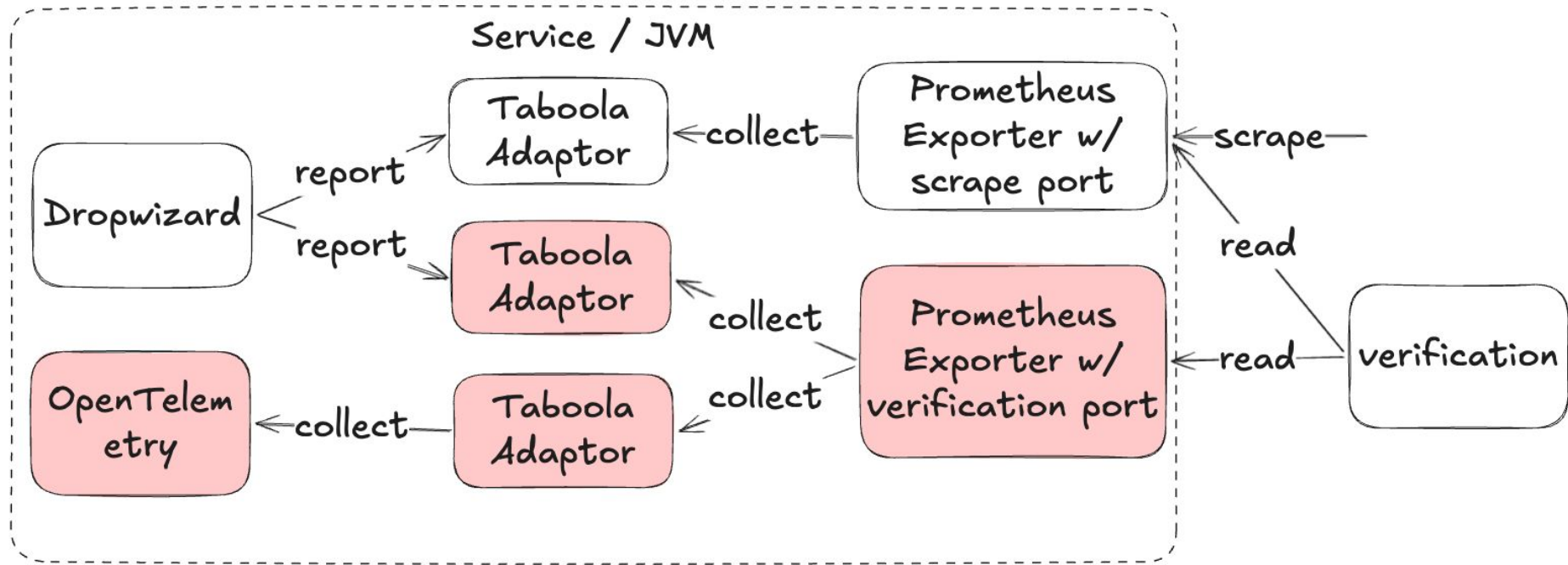
# Decision

- OpenTelemetry for new metrics going forward

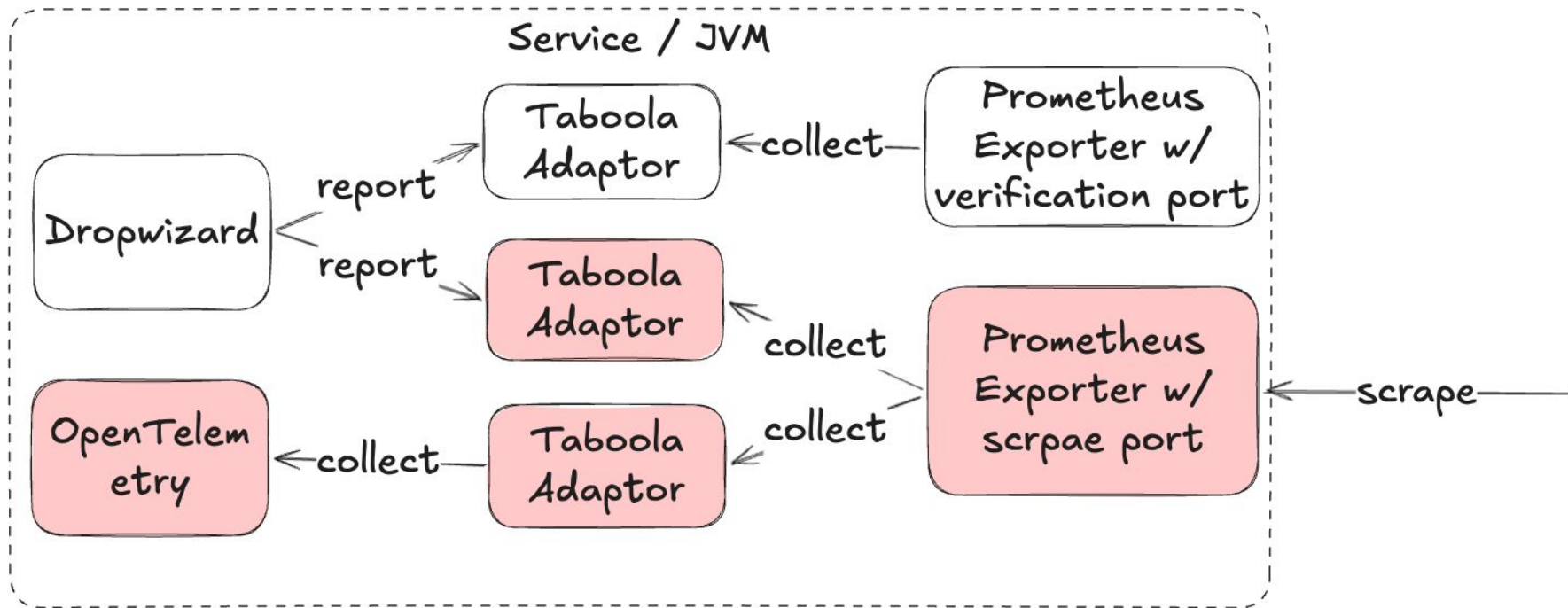
# Solution



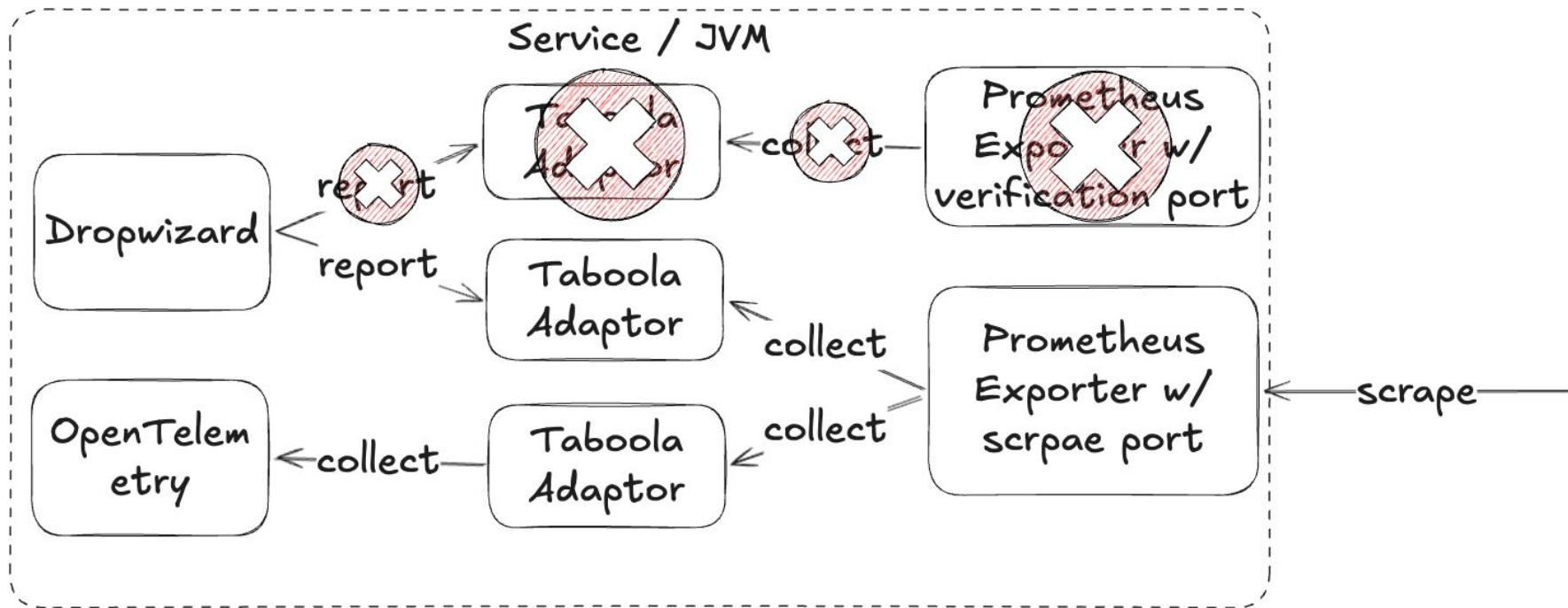
# Parallel verification



# Ease of rollback – Ports as configuration



# Full rollout



# Naming backward compatibility

- Prometheus new library enforces suffix convention
  - Counter: `_total`
  - Summary: `_count`, `_sum`, `` (empty for buckets)
  - Gauge: `` (empty)
- But we have own convention for legacy metrics
  - EX: `SERVICE_counter`

# Naming backward compatibility

- Solution – export all legacy metrics as Gauge, which has no suffix
- EX: `SERVICE_counter`
  - As counter: `SERVICE_counter_total`
  - As gauge: `SERVICE_counter`

# Friendly interface and caching

*// OpenTelemetry interface*

```
LongHistogram requestLatency1 = otelMeter.histogramBuilder("request_latency") DoubleHistogramBuilder
    .ofLongs() LongHistogramBuilder
    .setUnit(Unit.MILLISECONDS.getUnit())
    .build();
requestLatency1.record(
    10,
    Attributes.builder()
        .put("endpoint", "awesome")
        .put("status", "OK")
        .build());
```

*// Taboola interface*

```
TaboolaLongHistogram requestLatency2 = taboolaMeter.longHistogram("request_latency", Unit.MILLISECONDS, "endpoint", "awesome");
requestLatency2.record(10, "status", "OK");
```



# More Taboola flavor

- Common labels auto applied
  - EX: domain, env, service, ...
- Heathiness metrics
  - EX: `taboola_service_uptime_millis`

# More Taboola flavor

- High cardinality *soft* protection
- Metric blacklist
- Dynamic metric name mutation

# Release

-  Transparent migration (  )
-  Backward compatibility
-  Parallel verification
-  Ease of rollback

# Adoption

- Usage documents
- Slack announcement and promotion
- Internal tech talk
- @Deprecated annotation

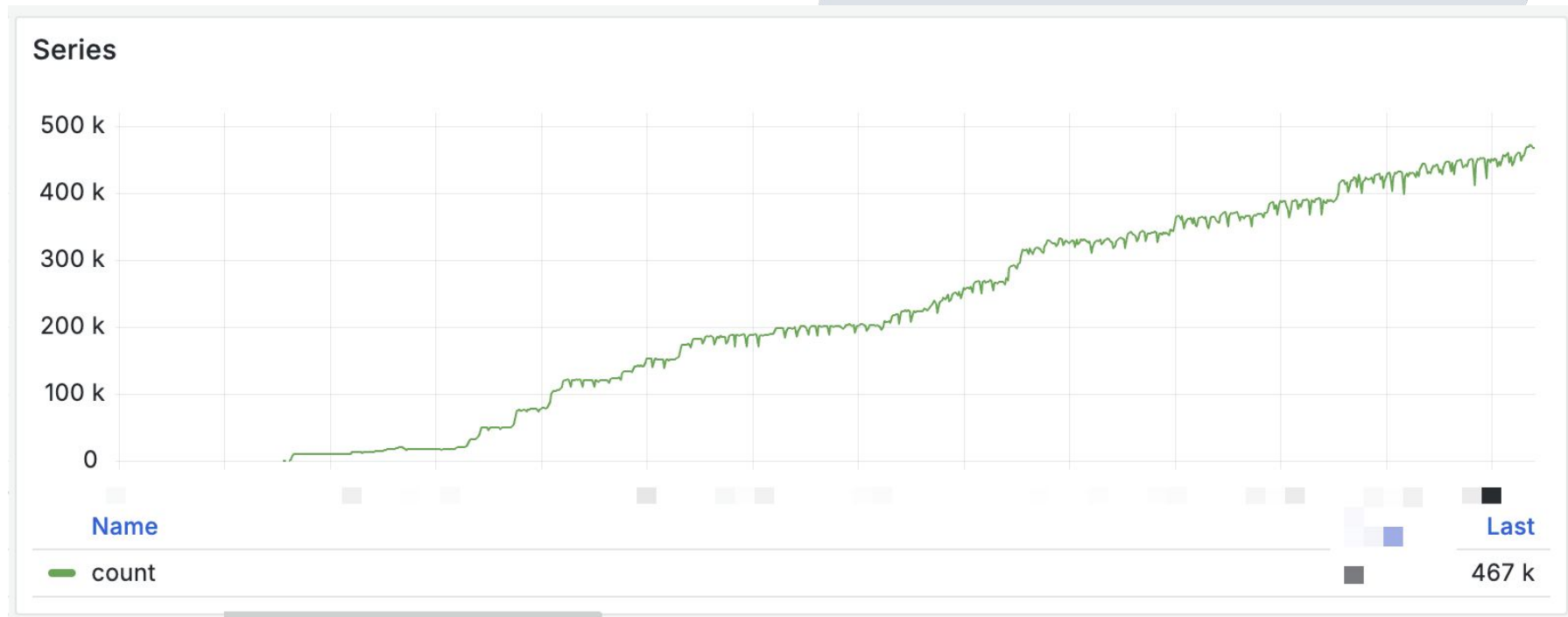
# Adoption

- Active suggestion when appropriate
- Friendly support on early adopters
- Quick iteration on feedback

# Adoption

- Automated PR review with deprecated metrics

# Usage in 3 months



**Thank You**